**LAPORAN PRAKTIKUM**

**BAB I, II, DAN III**



**Disusun untuk Memenuhi Tugas**

**Mata Kuliah:**

**Workshop DevOps**

**Dosen Pengampu :**

**Dr. Ferry Astika Saputra, ST, M.Sc**


**Anggota Kelompok 2:**

**Ibram Maulana Akhsanul Qasasi NRP. 3126640014**

**Yusuf Rahmadhani Asy'ari NRP. 3126640042**

**Vernanda Mulia Hamonangan Manurung NRP. 3126640044**



**PROGRAM STUDI D4 LANJUT JENJANG TEKNIK INFORMATIKA**

**POLITEKNIK ELEKTRONIKA NEGERI SURABAYA**

**2026**

# BAB I

# FONDASI TEORETIS DAN KERANGKA KERJA DEVSECOPS

## 1.1 Lingkungan Kerja (Environment)

Praktikum dilakukan pada Virtual Machine (VM) yang berjalan pada platform virtualisasi Proxmox VE. Lingkungan yang digunakan adalah sebagai berikut:

- Sistem Operasi: Ubuntu Linux 22.04 LTS
- Arsitektur: linux/amd64
- Hostname: ubn-2
- User: ubuntu-2
- IP Address: 10.252.108.103

Seluruh proses instalasi, konfigurasi, dan pengujian Docker dilakukan melalui terminal pada VM tersebut. Pengujian akses service container dilakukan melalui browser menggunakan IP address VM.

## 1.2 Langkah Kerja dan Hasil Implementasi

### 1.2.1 Menetapkan Baseline Laboratorium

Langkah awal dijalankan pada mesin host ubuntu-2@ubn-2 dengan membuat direktori kerja tunggal ~/devsecops-lab/ yang memiliki sub-direktori khusus terpisah sesuai peruntukannya. Kemudian lakukan pemeriksaan versi docker dilakukan untuk mencatat baseline environment.




## 1.3 Verifikasi dan Skenario Pengujian

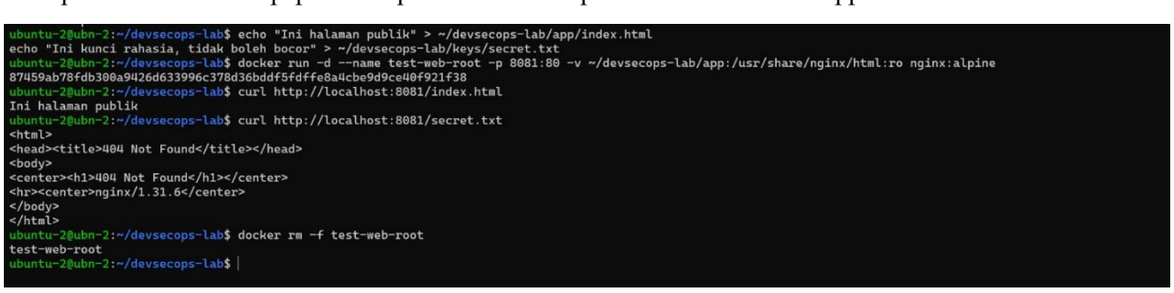
### 1.3.1 Verifikasi Akses Docker Daemon & Kontainer Aktif

Pengujian dilakukan untuk memastikan akun praktikum (ubuntu-2) dapat berkomunikasi langsung dengan Docker daemon tanpa perlunya perantara hak akses eksternal, serta memverifikasi kesiapan plugin Compose:




### 1.3.2 Pengujian Skenario Isolasi Web Root

Skenario pengujian dikembangkan untuk membuktikan bahwa direktori reports, sbom, dan keys aman dari akses publik dan tidak terpapar saat aplikasi web mempublikasikan direktori app/.



### 1.3.3 Pemeriksaan Fitur Keamanan Host Kernel

Opsi keamanan yang aktif pada host Linux diperiksa menggunakan format JSON untuk mengetahui profil pembatasan sistem (Mandatory Access Control):

```
ubuntu-2@ubn-2:~/devsecops-lab$ docker info --format '{{json .SecurityOptions}}'
["name=apparmor,profile=default","name=seccomp,profile=builtin","name=cgroupns"]
```

### 1.3.4 Threat Statement Praktikum Baseline

Aset utama yang dilindungi dalam lingkungan laboratorium ini mencakup kunci kriptografi dan berkas rahasia (~/devsecops-lab/keys/secret.txt), laporan hasil analisis kerentanan (reports/), kebijakan keamanan (policy/), serta dokumen *Software Bill of Materials* (sbom/). Aktor ancaman potensial meliputi penyerang eksternal yang melakukan pemindaian endpoint web secara proaktif maupun pengguna lokal berhak akses rendah (*unprivileged local user*) yang berada di dalam jaringan laboratorium. Jalur serangan (*attack vector*) dapat terjadi apabila pengembang melakukan kesalahan konfigurasi pemetaan volume (*volume bind mount misconfiguration*) dengan memetakan seluruh direktori induk ~/devsecops-lab/ sebagai *document root* web server, atau akibat ditemukannya celah pemutaran direktori (*Directory Traversal / Path Traversal*) pada aplikasi web. Dampak dari terwujudnya ancaman ini adalah kebocoran kredensial dan kunci rahasia secara publik (*Confidentiality Breach*), yang memberikan jalan bagi penyerang untuk melakukan eskalasi hak akses (*Privilege Escalation*), mengambil alih kendali infrastruktur kontainer, serta memalsukan laporan kepatuhan keamanan laboratorium.

## 1.4 Evaluasi dan Latihan Mandiri

1. Mengapa DevSecOps tidak dapat direduksi menjadi penambahan scanner pada pipeline?

DevSecOps merupakan transformasi holistik yang mencakup budaya, proses kerja, dan teknologi (*people, process, technology*), bukan sekadar otomatisasi pengujian perangkat lunak. Penambahan alat pemindai (seperti SAST, DAST, atau SCA) ke dalam pipeline CI/CD tanpa integrasi budaya dan kebijakan perbaikan (*remediation SLAs*) hanya akan menghasilkan kebisingan sinyal berupa *false positives* dan pemicu *alert fatigue* bagi tim pengembang.

DevSecOps menuntut pergeseran paradigma tanggung jawab keamanan ke tahap paling awal (*Shift-Left Security*), penyusunan kebijakan keamanan terstruktur sebagai kode (*Policy-as-Code*), manajemen risiko yang berkelanjutan, serta kolaborasi aktif antara tim pengembang, operasi, dan sekuritas. Scanner hanyalah instrumen deteksi pasif; tanpa adanya tindak lanjut remedi, tata kelola (*governance*), dan peningkatan kapasitas (*security awareness*) pengembang, keberadaan scanner tidak otomatis menjamin pipeline menjadi aman.

2. Evidence apa yang membedakan klaim kontrol dari kontrol yang benar-benar terverifikasi?

Klaim kontrol bersifat deklaratif dan konseptual (misalnya pernyataan "sistem kami telah mengisolasi direktori sensitif dan menggunakan konfigurasi kernel aman") tanpa menyertakan bukti empiris. Sebaliknya, kontrol yang benar-benar terverifikasi (*verified controls*) dibuktikan melalui artifak dan rekaman data objektif yang dapat diuji ulang (*reproducible*).

Bukti nyata (*verifiable evidence*) yang membedakan keduanya mencakup:

- Log Eksekusi Pengujian Nyata: Hasil tangkapan respon HTTP (seperti curl http://localhost:8081/secret.txt yang mengembalikan status 404 Not Found) yang membuktikan isolasi *web root* bekerja secara efektif.
- Rekaman Baseline & Konfigurasi Sistem: Data mentah keluaran sistem seperti docker info --format '{{json .SecurityOptions}}' yang membuktikan aktifnya profil *AppArmor*, *Seccomp*, dan *Cgroupns*.
- Artifak Audit Terstruktur: Dokumen *Software Bill of Materials* (SBOM), berkas laporan audit keamanan berbasis format standar (SARIF/JSON), serta rekaman pencatatan versi komponen yang presisi (Docker 29.8.1, Git 2.43.0, OpenSSL 3.0.13).

3. Bagaimana shared responsibility memengaruhi ownership risiko dan tindak lanjut temuan?

Model Tanggung Jawab Bersama (*Shared Responsibility Model*) mengeliminasi batas-batas *silo* tradisional di mana tim keamanan (*Security*) sering dikambinghitamkan sebagai satu-satunya pemilik risiko sistem. Model ini membagi kepemilikan risiko dan tanggung jawab remedi secara proporsional sesuai dengan domain kontrolnya:

- Ownership Risiko (Kepemilikan Risiko): Tim Pengembang (*Developers*) memiliki risiko atas kode aplikasi yang ditulis dan dependensi pustaka yang digunakan (*application layer risk*). Tim Operasi (*Ops/Platform*) bertanggung jawab atas keamanan infrastruktur, sistem operasi, dan konfigurasi runtime kontainer (*infrastructure layer risk*). Tim Keamanan (*Security*) mengelola risiko tingkat kebijakan, pemodelan ancaman, dan standar kepatuhan organisasi.
- Tindak Lanjut Temuan: Temuan kerentanan tidak lagi sekadar "dilemparkan" oleh tim sekuritas ke pengembang. Setiap temuan memiliki jalur perbaikan yang terdefinisi dengan SLA yang terukur. Tim pengembang bertanggung jawab memperbarui pustaka atau memperbaiki cacat logika kode, sementara tim sekuritas bertindak sebagai *enabler* yang menyediakan panduan remedi, alat pemindaian otomatis, dan verifikasi ulang hasil tindakan remedi.

# BAB II

# KONSEP CONTAINER DAN INSTALASI DOCKER

## 2.2 Langkah Kerja dan Hasil Implementasi

### 2.2.1 Inisialisasi Direktori dan Workspace

Langkah awal praktikum dimulai dengan menyiapkan direktori kerja yang terisolasi untuk mengorganisir seluruh file konfigurasi dan dependensi praktikum Docker Bab 02. Perintah mkdir -p digunakan untuk membuat direktori secara bertingkat.

```
ubuntu-2@ubn-2: $ mkdir -p ~/docker-lab/bab-2
ubuntu-2@ubn-2: $ ls
docker-lab
ubuntu-2@ubn-2: $ cd ~/docker-lab/bab-2
```

***Gambar 1.1 Pembuatan direktori kerja ~/docker-lab/bab-2 dan navigasi workspace.***

Direktori docker-lab/bab-2 berhasil dibuat di bawah direktori home user. Pengujian dengan perintah ls mengonfirmasi keberadaan direktori, dan perintah cd mengarahkan sesi terminal aktif ke dalam lokasi direktori tersebut.

### 2.2.2 Instalasi Paket Dependensi dan Direktori Keyrings

Sebelum menambahkan repository resmi Docker Engine, sistem perlu diperbarui dengan paket dependensi dasar yang dibutuhkan untuk komunikasi jaringan HTTPS, manajemen sertifikat, serta kunci enkripsi GPG.

```
ubuntu-2@ubn-2:~/docker-lab/bab-2$ sudo apt install -m ca-certificates curl gnupg lsb-release
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20260601~24.04.1).
ca-certificates set to manually installed.
curl is already the newest version (8.5.0-2ubuntu10.13).
curl set to manually installed.
gnupg is already the newest version (2.4.4-2ubuntu17.6).
gnupg set to manually installed.
lsb-release is already the newest version (12.0-2).
lsb-release set to manually installed.
The following packages were automatically installed and are no longer required:
  libfwupd2 libgusb2
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 9 not upgraded.
ubuntu-2@ubn-2:~/docker-lab/bab-2$ sudo install -m 0755 -d /etc/aptkeyrings
```

***Gambar 1.2 Pengunduhan paket ca-certificates, curl, gnupg, lsb-release, dan pembuatan folder keyrings.***

Paket ca-certificates, curl, gnupg, dan lsb-release terverifikasi telah berada pada versi terbaru (newest version). Direktori /etc/apt/keyrings berhasil dibuat dengan izin akses 0755 untuk menyimpan kunci publik GPG.

### 2.2.3 Konfigurasi GPG Keyring dan Official Repository Docker

Untuk menjamin integritas dan autentisitas paket perangkat lunak Docker yang diunduh via APT, kunci GPG resmi Docker diunduh dan disimpan ke dalam direktori keyrings, kemudian entri repository ditambahkan ke dalam skema paket APT Ubuntu.

```
ubuntu-2@ubn-2:~/docker-lab/bab-2$ sudo chmod a+r /etc/apt/keyrings/docker.gpg
ubuntu-2@ubn-2:~/docker-lab/bab-2$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(lsb_release
 -cs) stable" |   sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

***Gambar 2.1 Pembaruan izin baca kunci GPG dan penambahan repositori resmi Docker ke APT sources.***

Berkas docker.gpg berhasil diatur dengan hak akses publik baca (a+r). Penambahan entri sumber daya APT pada /etc/apt/sources.list.d/docker.list memastikan sistem mendapatkan pembaharuan Docker Engine secara langsung dari repositori resmi.

### 2.2.4 Pengaturan Hak Akses User Non-Root

Secara default, Docker Daemon berjalan di bawah hak akses akun root. Agar pengguna biasa dapat menjalankan perintah Docker tanpa menyertakan instruksi sudo pada setiap eksekusi, user ditambahkan ke dalam Unix group docker.

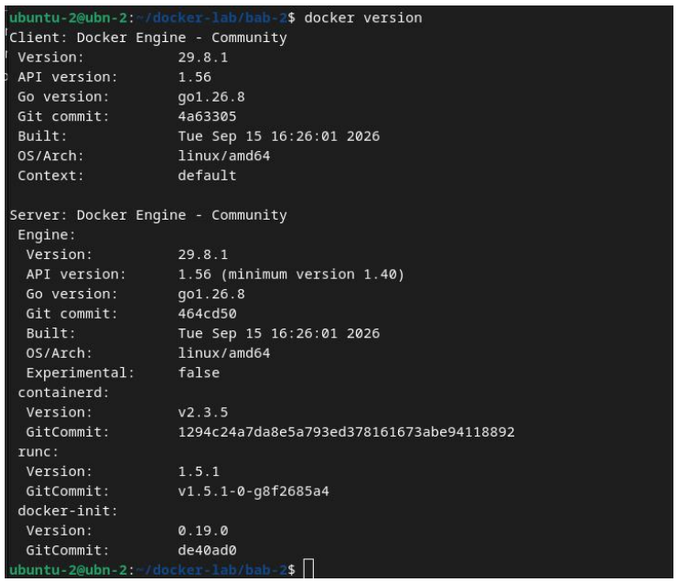
```
ubuntu-2@ubn-2:~/docker-lab/bab-2$ sudo usermod -aG docker $USER
ubuntu-2@ubn-2:~/docker-lab/bab-2$ newgrp docker
```

***Gambar 2.2 Penambahan user aktif ke unix group docker dan penyegaran hak akses via newgrp.***

Akun pengguna ubuntu-2 berhasil dimasukkan ke dalam grup docker. Penggunaan perintah newgrp docker merefresh keanggotaan grup dalam sesi shell aktif tanpa memerlukan re-login sistem.

### 2.2.5 Verifikasi Instalasi dan Status Docker Engine

Guna memastikan bahwa Docker Daemon telah aktif dan dapat berkomunikasi dengan CLI client secara normal, dilakukan pengecekan arsitektur dan versi komponen Docker Engine.

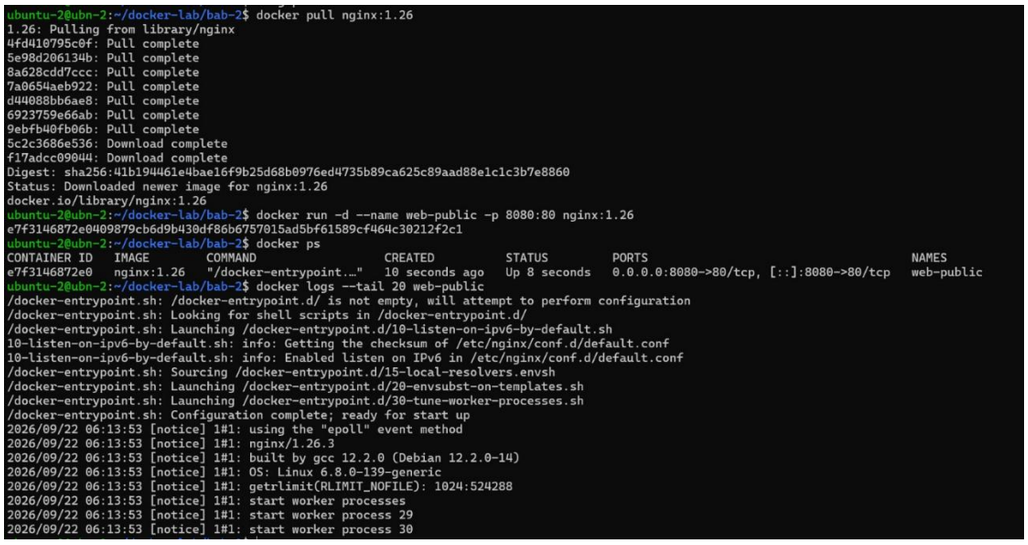



***Gambar 2.3 Verifikasi versi Docker Engine (Client v29.8.1 & Server v29.8.1, containerd v2.3.5).***

Perintah docker version menampilkan informasi lengkap mengenai Client dan Server Docker Engine Community versi 29.8.1, API version 1.56, runtime runc v1.5.1, serta supervisor containerd v2.3.5 yang menandakan instalasi berhasil.

### 2.2.6 Deployment dan Monitoring Container Web Server (nginx)

Praktikum dilanjutkan dengan mengunduh (pull) image resmi Nginx versi 1.26 dari Docker Hub, kemudian menjalankan container dalam mode terisolasi di latar belakang (detached mode -d) serta memetakan port host 8080 ke port 80 milik container.




***Gambar 3.1 Pengunduhan image Nginx 1.26, eksekusi container web-public, dan pemeriksaan log proses.***

Image nginx:1.26 berhasil diunduh. Container dinamai web-public dan berstatus Up. Log internal mengonfirmasi bahwa worker processes Nginx telah berjalan.

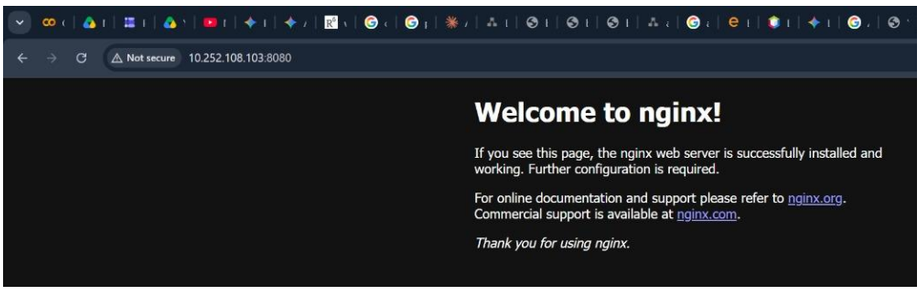
### 2.2.7 Validasi Pengujian Web Server Via CLI dan Browser

Untuk memverifikasi konektivitas layanan web yang berjalan di dalam container, pengujian dilakukan melalui dua pendekatan: panggilan HTTP menggunakan command curl dan pengujian antarmuka melalui Web Browser.

```
ubuntu-2@ubn-2:~/docker-lab/bab-2$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```




***Gambar 3.2 Tampilan verifikasi halaman web Nginx "Welcome to nginx!" yang diakses via browser pada IP 10.252.108.103:8080.***

Hasil command curl http://localhost:8080 mengembalikan struktur dokumen HTML resmi Nginx. Pengujian lanjutan via Google Chrome pada alamat IP Host http://10.252.108.103:8080 menampilkan halaman pembuka Nginx.

### 2.2.8 Eksplorasi Container Interaktif

Docker memungkinkan pembentukan lingkungan sistem operasi terisolasi secara cepat. Command pada gambar 4.1 digunakan untuk masuk ke dalam sesi terminal interaktif container Ubuntu.

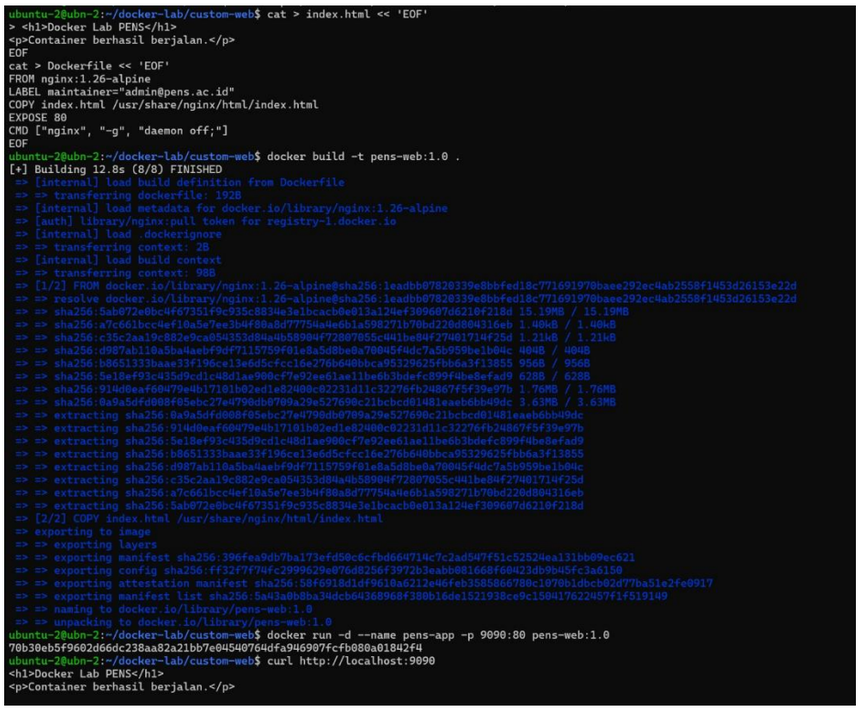
```
ubuntu-2@ubn-2:~/docker-lab/bab-2$ docker run -it --name ubuntu-test ubuntu:22.04 /bin/bash
Unable to find image 'ubuntu:22.04' locally
22.04: Pulling from library/ubuntu
20c3783cc497: Pull complete
dfa215c03b9b: Download complete
Digest: sha256:b8b6ee6aa931ecd9d0d952abc34dc0e5f7c6a30c6bb71b079fe399fde0329c02
Status: Downloaded newer image for ubuntu:22.04
root@932486173c62:/# cat /etc/os-release
PRETTY_NAME="Ubuntu 22.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.5 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy
root@932486173c62:/#
```

***Gambar 4.1 Pengujian respon curl localhost dan eksekusi container interaktif Ubuntu 22.04 LTS.***

Command tersebut berhasil berjalan dan terhubung dengan akun root@932486173c62:/#. Pengecekan file /etc/os-release mengonfirmasi bahwa distribusi yang berjalan di dalam container adalah Ubuntu 22.04.5 LTS.

### 2.2.9 Pembuatan Custom Image Menggunakan Dockerfile

Praktikum dilanjutkan dengan membangun Custom Docker Image berbasis nginx:1.26-alpine yang memuat halaman web kustom ciptaan sendiri melalui instruksi spesifik pada berkas Dockerfile.




***Gambar 4.2 Pembuatan Dockerfile, proses docker build pens-web:1.0, eksekusi pens-app port 9090, dan pengujian curl.***

Proses build image pens-web:1.0 berhasil. Container pens-app di-deploy pada port host 9090, dan output curl http://localhost:9090 menampilkan tag HTML kustom dengan benar.

### 2.2.10 Pengujian Custom Web Server Via Browser dan Verifikasi Skenario

Untuk membuktikan ketersediaan custom web application bagi pengguna luar, pengujian dilakukan dengan membuka antarmuka browser pada alamat IP Host http://10.252.108.103:9090.




***Gambar 5.1 Pengujian akses browser ke Custom Web Server pens-web:1.0 melalui IP Host 10.252.108.103:9090.***

### 2.2.11 Matriks Verifikasi Dan Skenario Pengujian

| Status | Kriteria Pengujian Modul | Bukti & Catatan Evaluasi |
|---|---|---|
| [✓] PASSED | Docker Engine aktif dan docker version menampilkan Client serta Server. | Terverifikasi pada Gambar 2.3 (Client v29.8.1 & Server v29.8.1 aktif). |
| [✓] PASSED | User non-root dapat menjalankan docker ps tanpa sudo. | Terverifikasi pada Gambar 2.2 & 3.1 (User ubuntu-2 menggunakan group docker). |
| [✓] PASSED | Container Nginx dapat diakses dari browser melalui port host. | Terverifikasi pada Gambar 3.2 (Akses 10.252.108.103:8080 sukses). |
| [✓] PASSED | Image pens-web:1.0 berhasil dibangun dan dijalankan. | Terverifikasi pada Gambar 4.2 & 5.1 (Akses 10.252.108.103:9090 sukses). |
| [✓] PASSED | Mahasiswa dapat menjelaskan perbedaan EXPOSE dan -p. | Teranalisis lengkap pada Bagian 12 laporan ini. |

### 2.2.12 Perbedaan Expose dengan -p

1. Instruksi EXPOSE (Metadata pada Dockerfile):

Instruksi EXPOSE 80 di dalam Dockerfile berfungsi sebagai dokumentasi internal / deklarasi metadata yang menginformasikan port mana yang digunakan oleh aplikasi di dalam container. Instruksi ini TIDAK membuka port ke antarmuka jaringan host luar secara otomatis dan tidak mengonfigurasi firewall / routing host.

2. Parameter -p / --publish (Docker CLI Runtime):

Parameter -p 9090:80 pada saat perintah docker run berfungsi secara aktif melakukan Pemetaan Port (Port Binding). Docker Daemon secara langsung memodifikasi aturan iptables / IP forwarding pada sistem operasi host untuk meneruskan seluruh lalu lintas dari port fisik host (9090) ke port internal container (80).

### 2.3 Evaluasi dan Latihan Mandiri

1. Mengapa penggunaan taglatesttidak dianjurkan untuk deployment yang harus reproducible?

Tag latest bersifat mutable (dapat berubah seiring waktu). Tag ini hanyalah sebuah pointer dinamis yang menunjuk pada build versi paling akhir yang diunggah ke registry. Apabila digunakan dalam skenario reproducible deployment (misalnya di lingkungan staging atau production), instruksi docker pull image:latest di kemudian hari berisiko mengunduh versi image yang telah diperbarui oleh developer. Hal ini dapat menimbulkan ketidakcocokan dependensi (dependency mismatch), breaking changes, atau perilaku aplikasi yang inkonsisten antar lingkungan. Untuk menjamin sifat reproducible, deployment wajib menggunakan tag versi spesifik (misal nginx:1.26.3) atau mereferensikan SHA256 Image Digest (misal image@sha256:41b19446...) yang bersifat immutable.

2. Jelaskan perancontainerddanruncdalam arsitektur Docker.

• runc (Low-Level Container Runtime): Komponen tingkat paling bawah yang mengimplementasikan spesifikasi OCI (Open Container Initiative). Tugas utamanya adalah berinteraksi langsung dengan kernel Linux untuk membuat, mengonfigurasi, dan menjalankan container process memanfaatkan fitur namespaces (isolasi PID, jaringan, mount point) dan cgroups (pembatasan memori/CPU). Setelah container selesai diinisialisasi, proses runc langsung keluar (exit).

• containerd (High-Level Container Runtime Daemon): Bertindak sebagai pengelola siklus hidup container (container lifecycle manager) tingkat menengah. containerd mengendalikan eksekusi runc, mengelola penarikan/penyimpanan image dari registry, mengatur sistem penyimpanan layer (OverlayFS), mengelola attachment jaringan dasar, serta memantau kesehatan proses container.

3. Apa konsekuensi keamanan dari memasukkan user ke groupdocker?

Memasukkan pengguna biasa (non-root) ke dalam grup docker memberikan hak akses yang setara dengan akses root penuh tanpa kata sandi (passwordless root privilege escalation) pada sistem host. Hal ini terjadi karena Docker Daemon berjalan dengan privilese root dan berkomunikasi melalui socket Unix /var/run/docker.sock yang diizinkan untuk diakses oleh anggota grup docker. User dalam grup ini dapat dengan mudah melakukan mount direktori root host ke dalam container (misal: docker run -v /:/host-root -it ubuntu bash), yang memungkinkan pengguna membaca/memodifikasi file sensitif sistem host seperti /etc/shadow atau /etc/sudoers. Praktik DevSecOps merekomendasikan penggunaan Rootless Docker Mode atau pembatasan ketat pengguna grup ini.

4. Bandingkan layer imagenginx:1.26-alpinedan image custom yang Anda buat.

Image base nginx:1.26-alpine berukuran sangat ringkas (~10 MB) dan terdiri dari layer OS minimalis Alpine Linux serta binaries Nginx. Saat membangun image custom pens-web:1.0 menggunakan Dockerfile, Docker menggunakan mekanisme Union File System (Overlay2). Image custom membagikan (sharing) seluruh layer read-only milik base image nginx:1.26-alpine tanpa menduplikasi data, dan hanya menambahkan 1 layer read-only baru yang berisi instruksi COPY index.html. Pendekatan layered storage ini sangat efisien dalam menghemat ruang penyimpanan disk dan mempercepat distribusi build.

5. Kapan sebaiknya memilih Virtual Machine (VM) daripada container?

Virtual Machine (VM) sebaiknya dipilih dibanding Container dalam kondisi: (1) Isolasi Keamanan Tingkat Tinggi (Strong Isolation): VM menggunakan hardware-level virtualization (Hypervisor) dengan Kernel OS terpisah, sehingga sangat cocok untuk lingkungan multi-tenant di mana antar entitas tidak boleh membagikan Kernel yang sama. (2) Kebutuhan OS / Kernel Berbeda: Ketika aplikasi membutuhkan Kernel atau OS yang berbeda dari host (misal menjalankan VM Windows pada Host Linux). (3) Aplikasi Legacy / Driver Khusus: Aplikasi terdahulu yang membutuhkan modul kernel spesifik, konfigurasi memori langsung, atau akses hardware device tertentu.

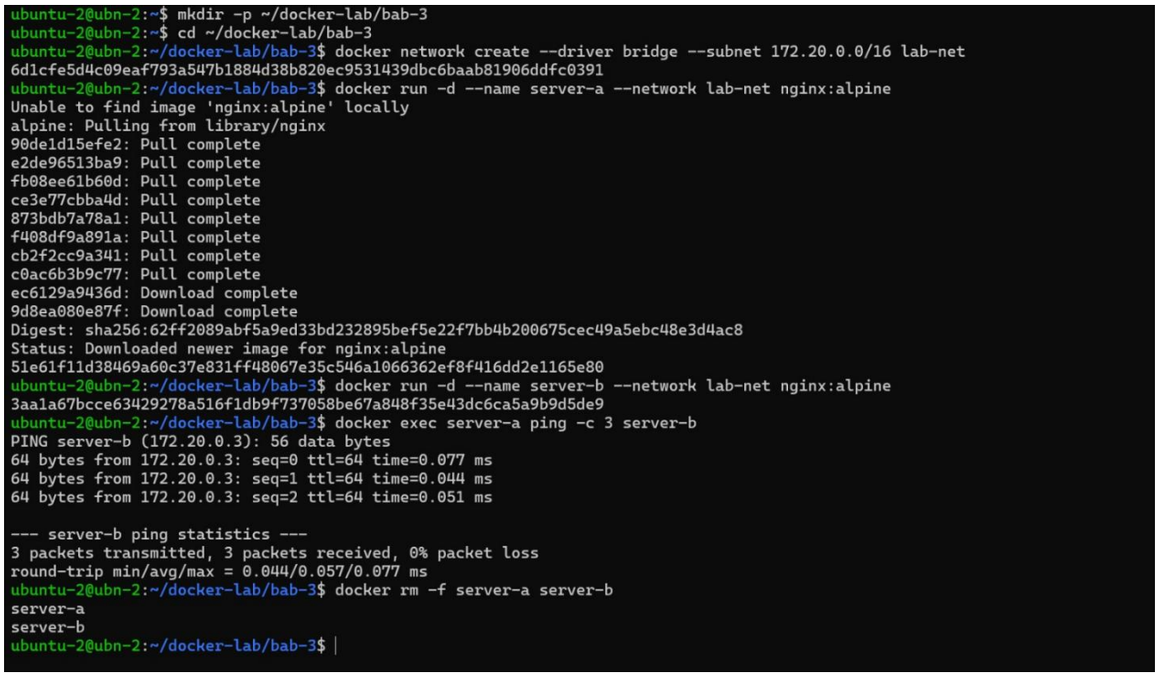
# BAB III

# DOCKER NETWORK, VOLUME, BIND MOUNT, TMPFS DAN COMPOSE

## 3.2 Langkah Kerja dan Hasil Implementasi

### 3.2.1 User-Defined Bridge Network

Mengonfigurasi jaringan bridge terisolasi kustom dengan subnet 172.20.0.0/16 untuk menguji ketersediaan fitur *Embedded DNS Server* internal Docker (127.0.0.11) dalam melakukan penyelesaian nama hostname antar-container secara otomatis.

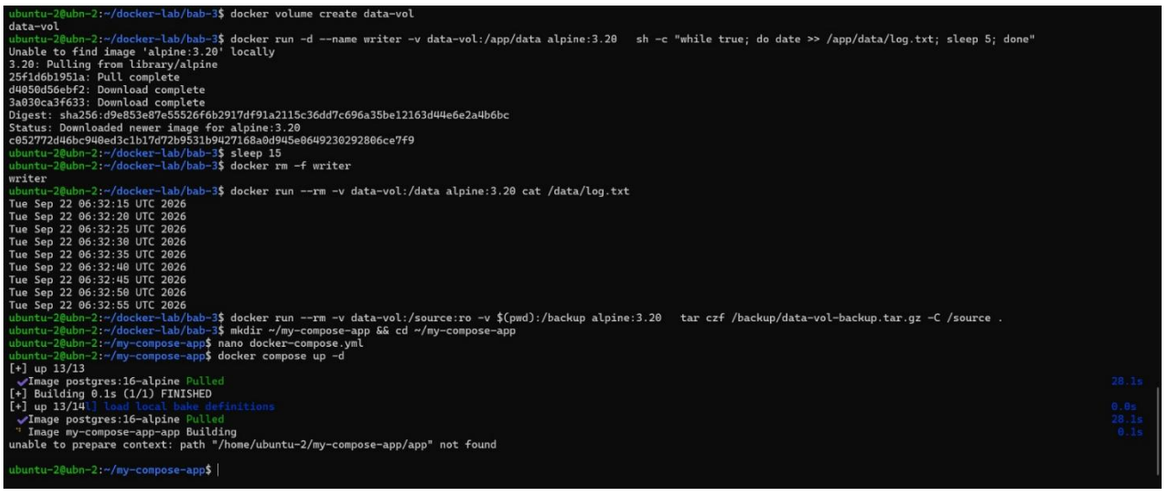



***Gambar 1.1 Pembuatan jaringan lab-net, deployment server-a & server-b, serta hasil ping sukses menggunakan hostname.***

Container server-a berhasil mengirim paket ICMP ping ke server-b (IP 172.20.0.3) menggunakan nama alias container secara langsung dengan 0% packet loss. Hal ini membuktikan bahwa jaringan bridge buatan user mengaktifkan automatic DNS discovery bawaan.

### 3.2.2 Volume Backup dan Restore

Pengujian daya tahan data menggunakan *Named Volume* dengan menuliskan catatan log waktu berkala, dilanjutkan dengan penghapusan container penulis dan pengarsipan backup berbasis tar.

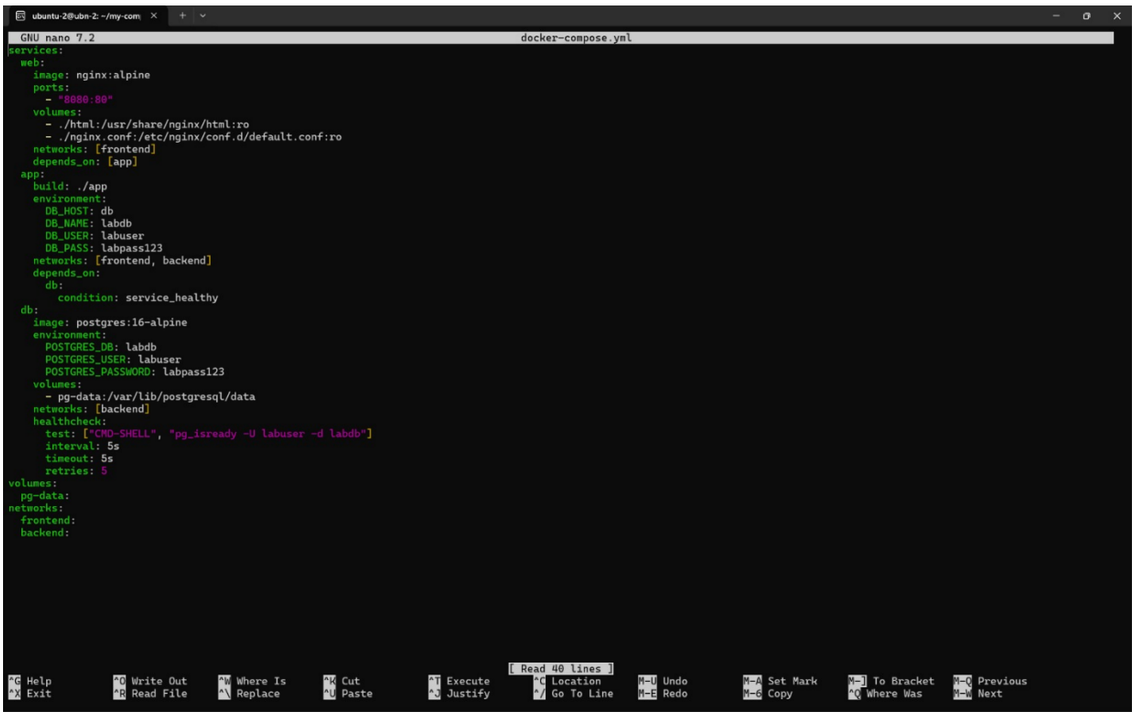



***Gambar 1.2 Verifikasi integritas data log setelah penghapusan container writer dan kompresi file backup tar.gz.***

Data log berturut-turut yang dibuat oleh container writer tetap dapat dibaca secara utuh oleh container Alpine baru setelah container asal dihancurkan. Proses backup tar czf juga berhasil mengompresi isi volume ke host.

### 3.2.3 Compose multi-container Nginx-Flask-PostgreSQL

Penyusunan berkas deklaratif docker-compose.yml yang mengoordinasikan tiga microservices: Nginx Web Proxy (web), Python Backend API (app), dan PostgreSQL Database (db) dilengkapi pengujian healthcheck.




***Gambar 1.3 Editor Nano menampilkan manifes docker-compose.yml dengan environment, volumes, & healthcheck.***

## 3.3 Verifikasi dan Skenario Pengujian Mandiri

### 3.3.1 Service Discovery pada User-Defined Bridge (my-net)

```
ubuntu-2@ubn-2:~/docker-lab/bab-3$ docker network create my-net
01dc7596d5f5dbce6ae2681c552f097873dd379120a75a91f12db7d196f96c11
ubuntu-2@ubn-2:~/docker-lab/bab-3$ docker run -d --name box1 --network my-net alpine sleep 3600
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
6d0606d1815c: Download complete
797dd00a0fc7: Download complete
Digest: sha256:294b683cb724975bec92580e1e685676bd4b50bda910ddb8c51d4cabeaec77e6
Status: Downloaded newer image for alpine:latest
79ced29bffcedfdb8ad2b7d2e1e85c3c4dfbc18b3d4624ac31a8820d4dffc9b2
ubuntu-2@ubn-2:~/docker-lab/bab-3$ docker run -it --rm --network my-net alpine ping -c 2 box1
PING box1 (172.18.0.2): 56 data bytes
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.159 ms
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.039 ms

--- box1 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.039/0.099/0.159 ms
```

***Gambar 3.1 Pengujian ping dari container ephemeral ke box1 (172.18.0.2) via nama alias pada jaringan my-net.***

Perintah pada Gambar 3.1 membuktikan bahwa container terpisah yang terhubung pada bridge network yang sama dapat saling menemukan IP melalui nama container tanpa bantuan konfigurasi DNS eksternal.

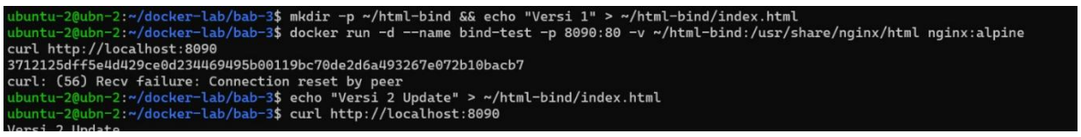
### 3.3.2 Persistensi Named Volume Setelah Container Removal

```
ubuntu-2@ubn-2:~/docker-lab/bab-3$ docker volume create data-vol
data-vol
ubuntu-2@ubn-2:~/docker-lab/bab-3$ docker run -d --name vol-test -v data-vol:/app/data alpine sh -c "echo 'Hello Proxmox' > /app/data/test.txt"
856346d3ea5639fa8d9644c5068f30d1e85ae0a4084cbde630f3dd264b29ee6c
ubuntu-2@ubn-2:~/docker-lab/bab-3$ docker rm -f vol-test
vol-test
ubuntu-2@ubn-2:~/docker-lab/bab-3$ docker run --rm -v data-vol:/app/data alpine cat /app/data/test.txt
Hello Proxmox
ubuntu-2@ubn-2:~/docker-lab/bab-3$
```

***Gambar 2.3 Pembacaan kembali teks Hello Proxmox dari data-vol setelah container vol-test dihapus.***

Data yang ditulis ke /app/data/test.txt berada di direktori khusus host (/var/lib/docker/volumes/). Saat container vol-test di-*remove* secara paksa, siklus hidup volume tetap aman dan datanya dapat diakses container baru.

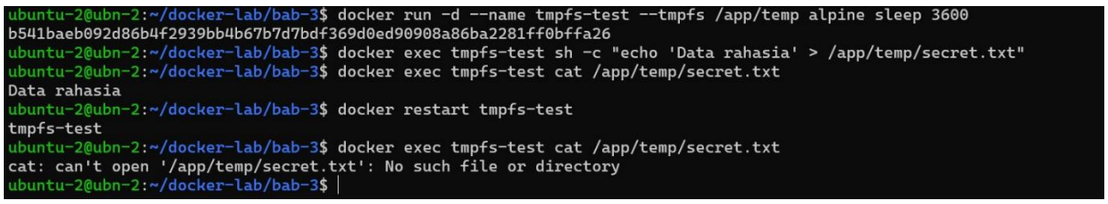
### 3.3.3 Live Development Hot-Reloading via Bind Mount




***Gambar 3.3 Respon curl berubah seketika menjadi Versi 2 Update saat file index.html di host dimodifikasi.***

Bind Mount mengikat direktori host ~/html-bind secara langsung ke web root Nginx /usr/share/nginx/html. Perubahan kode di host langsung berdampak pada respon HTTP container tanpa perlu rebuild image.

### 3.3.4 Pengujian Storage Volatile / In-Memory (tmpfs)




***Gambar 3.4 File secret.txt terhapus setelah container tmpfs-test di-restart.***

Storage tmpfs dialokasikan pada RAM host. Ketika container di-restart, memori dibebaskan sehingga seluruh file di dalam mount point /app/temp terhapus permanen demi keamanan data sensitif.

### 3.3.5 Multi-Container Deployment & Healthcheck Verification




***Gambar 3.5 Output docker compose ps menunjukkan status Up dan Database berstatus (healthy).***

Eksekusi command pada Gambar 3.5 berhasil menjalankan ketiga layanan. Fitur Healthcheck secara konsisten memverifikasi kesiapan database PostgreSQL sebelum menerima traffic.

### 3.3.6 Matriks Verifikasi Dan Skenario Pengujian

| No | Item Pengujian Verifikasi | Status | Bukti & Observasi Teknis |
|---|---|---|---|
| 1 | Container di user-defined bridge dapat saling resolve via nama. | [✓] PASSED | Gambar 3.1: Ping ke server-b dan box1 sukses 0% loss via Embedded DNS. |
| 2 | Data di named volume tetap ada setelah container dihapus. | [✓] PASSED | Gambar 3.2: Data log & string 'Hello Proxmox' di data-vol dibaca utuh oleh container baru. |
| 3 | Bind mount menunjukkan perubahan file host tanpa rebuild. | [✓] PASSED | Gambar 3.3: Output curl menampilkan 'Versi 2 Update' secara real-time dari host. |
| 4 | tmpfs kehilangan data setelah container restart. | [✓] PASSED | Gambar 3.4: File secret.txt hilang total dari RAM setelah perintah restart. |
| 5 | Compose stack berjalan dan API health menampilkan status db. | [✓] PASSED | Gambar 3.5: Seluruh container aktif dan database bernilai (healthy). |

### 3.3.7 Evaluasi dan Latihan Mandiri

1. Keunggulan User-Defined Bridge

*User-defined bridge* jauh lebih baik daripada *default bridge* untuk aplikasi *multi-container* karena menyediakan fitur *Automatic DNS Resolution*, sehingga container dapat saling berkomunikasi cukup menggunakan nama layanannya tanpa mengandalkan alamat IP yang dinamis. Selain itu, *user-defined bridge* memberikan isolasi jaringan yang lebih ketat karena hanya container dalam jaringan yang sama yang dapat terhubung, serta memungkinkan pengembang untuk menghubungkan atau melepas container dari jaringan secara langsung tanpa perlu melakukan *restart*.

2. Risiko Keamanan Bind Mount

Risiko utama dari *bind mount* adalah terpaparnya direktori dan berkas sensitif dari sistem host secara langsung ke dalam container. Jika container berhasil diretas atau dijalankan menggunakan akses *root*, penyerang dapat membaca, mengubah, atau menghapus berkas penting pada mesin host (seperti berkas konfigurasi sistem /etc atau kunci SSH). Hal ini dapat menyebabkan pembobolan batas isolasi (*container escape*) dan pengambilalihan kendali penuh atas server host.

3. Perbedaan docker compose down dan docker compose down -v

Perbedaan dasarnya terletak pada penanganan data *volume*. Perintah docker compose down hanya menghentikan serta menghapus container dan jaringan yang dibuat oleh Compose, tetapi tetap membiarkan *named volume* beserta datanya utuh di host. Sebaliknya, perintah docker compose down -v menghentikan dan menghapus container, jaringan, sekaligus menghapus seluruh named volume yang terdaftar pada file docker-compose.yml, sehingga seluruh data persisten (seperti basis data) akan terhapus secara permanen.

4. Penggunaan depends_on dengan healthcheck

Penggunaan depends_on yang dikombinasikan dengan healthcheck (condition: service_healthy) lebih tepat digunakan ketika suatu layanan membutuhkan layanan lain untuk benar-benar siap menerima transaksi data, bukan sekadar dalam status menyala. Sebagai contoh, aplikasi *backend* yang membutuhkan database; depends_on biasa hanya menunggu proses container database mulai berjalan, sedangkan kombinasi healthcheck memastikan database telah selesai melakukan inisialisasi dan siap menerima koneksi sebelum aplikasi *backend* mulai dinyalakan, sehingga terhindar dari error *connection refused*.

5. Strategi Backup Volume Database Produksi

Strategi *backup volume* database produksi dapat dilakukan dengan dua metode utama. Pertama, menggunakan skrip otomatis (*cron job*) yang menjalankan utilitas *dump* bawaan database (seperti pg_dump atau mysqldump) melalui perintah docker exec tanpa menghentikan layanan. Kedua, dengan metode *cold backup*, yaitu menghentikan sementara container database, lalu menjalankan *helper container* sementara yang melakukan *mount* pada *volume* database untuk mengarsipkan direktori data ke dalam file kompresi (.tar.gz). Seluruh berkas cadangan tersebut selanjutnya harus langsung dikirim dan disimpan ke lokasi penyimpanan terpisah seperti *Cloud Storage* (S3) untuk mengantisipasi kegagalan perangkat keras pada server utama.